

Organic Flow Web Uygulaması

Geliştirme ve İmplementasyon Planı (v1.0)

1. Mimari ve Teknoloji Yığını (Tech Stack)

Uygulamanın farklı bilgisayarlardan erişilebilir, eşzamanlı (real-time) çalışabilir ve görsel odaklı olması için modern bir web mimarisi kurulmalıdır.

Frontend (İstemci)

- Framework:** React.js veya Vue.js
- Harita/Akış Motoru:** **React Flow** (Büyütme/küçültme, sürükle-bırak ve oklarla bağlama işlemleri için sektör standartıdır).
- Durum Yönetimi:** Zustand veya Redux (kullanıcı giriş verilerini ve harita state'ini tutmak için).

Backend (Sunucu)

- Framework:** Python ile **FastAPI**. Yüksek performanslıdır, asenkron yapıyı destekler ve Python ekosistemindeki tecrübenle çok hızlı backend yazmanı sağlar.
- Gerçek Zamanlı İletişim:** WebSockets (Farklı bilgisayarlardaki kullanıcıların aynı anda haritadaki değişiklikleri görebilmesi için).

Veritabanı & Auth

- Veritabanı:** PostgreSQL (Kullanıcılar ve görevler için) + SQLAlchemy ORM.
- Kimlik Doğrulama:** JWT (JSON Web Tokens). Kullanıcı giriş yaptığında token alır ve tüm istekleri bu token ile yapar.

2. Veritabanı Tasarımı (Core Şema)

Sistem temelde 3 ana tablo üzerinde yükselecektir:

A. Users (Kullanıcılar)

Ekip arkadaşlarının kayıt olduğu tablo.

- `id` (UUID)
- `username` , `email` , `password_hash`
- `avatar_color` (Haritada hızlı tanınmaları için)

B. Nodes (Görev Hücreleri)

Haritadaki her bir kutucuk.

- `id` (UUID)
- `title` (Örn: Buton Ekle)
- `status` (TODO, IN_PROGRESS, BLOCKED_SOS, DONE)
- `assignee_id` (Görevi kim aldı? - Users tablosuna Foreign Key)
- `canvas_x` , `canvas_y` (Hücrenin haritadaki X ve Y koordinatları)
- `how_i_solved` (Kapanış wiki notu)
- `parent_node_id` (Küme/Odalar mantığı için - Hangi ana hücrenin içinde yer alıyor?)

C. Edges (Bağlantı Okları)

Görevlerin birbirini takip etme mantığı.

- `id` (UUID)
- `source_node_id` (Çıkan ok - Nodes tablosu FK)
- `target_node_id` (Giren ok - Nodes tablosu FK)

3. Geliştirme Aşamaları (Fazlar)

Faz 1: Temel İskelet (Kullanıcı ve Canvas)

- Backend'de kullanıcı kayıt (Register) ve giriş (Login) sisteminin JWT ile yazılması.
- Frontend'de boş bir "React Flow" tuvalinin ekrana basılması.
- Sağ tık (Context Menu) ile tuvale boş bir görev kutucuğu (Node) ekleme işlevinin yapılması.

Faz 2: Organik Büyüme (Dallanma ve İlişkiler)

- İki görev kutucuğu arasında ok (Edge) çizilebilmesi.
- Bir görev tıklandığında içinden alt görevlerin açılması (Progressive Disclosure mantığının uygulanması).
- Görev detay penceresinin yapılması (Durum değiştirme, görevi üzerine alma).

Faz 3: Çok Oyunculu Mod (WebSockets)

- FastAPI üzerinden WebSocket endpoint'i açılması.
- Ali kendi bilgisayarında görevi "Tıkandım (SOS)" durumuna getirdiğinde, tetiklenen WebSocket sinyali ile Ayşe'nin bilgisayarındaki ekranın anında güncellenmesi ve o hücrenin kırmızı yanması.
- Fare tekerleği ile zoom yapıldığında React Flow'un "Zoom Level" verisine göre kutucuk içindeki detaylı yazıların gizlenmesi/gösterilmesi.

Faz 4: Mini-Wiki ve Kapanış

- Görevi "Bitti" sütununa/durumuna çeken kullanıcıya "Nasıl Çözdün?" modal'ının çıkartılması.
- Bu verinin veritabanında `how_i_solved` sütununa kaydedilmesi ve hücrenin kalıcı olarak yeşile dönmesi.

4. Kritik Zorluklar ve Çözümleri

- **Durum Senkronizasyonu:** İki kişi aynı anda aynı kutucuğu düzenlemeye çalışırsa veri çakışması olur. *Çözüm:* Biri kutuya tıkladığında WebSocket üzerinden o kutuyu geçici olarak "Kilitli (Editing by X)" statüsüne alın.
- **Sonsuz Canvas Performansı:** Çok fazla görev olduğunda tarayıcı kasabilir. *Çözüm:* Harita üzerinde ekranın dışında kalan düğümleri (viewport dışı) render etmeyen sanallaştırma (virtualization) teknikleri kullanın.